REQUIREMENTS SPECIFICATION

Fiberpro ERP

with AI Agent Harness

A modern web rebuild of the Fiberpro VB.NET textile / garment ERP,

with an integrated AI agent that controls every module via natural-language prompts.

Document Type: Software Requirements Specification (SRS)
Version: 1.0
Status: Draft for Review
Date: August 24, 2026
Author: Fiberpro Rebuild Project
Audience: Engineering, Product, QA, Operations

Table of Contents

Right-click the table of contents → "Update Field" to refresh page numbers.

1. Executive Summary

Fiberpro ERP is a comprehensive textile and garment enterprise resource planning system originally developed as a Windows desktop application in VB.NET with a Microsoft SQL Server backend, Crystal Reports for printed documents, and a multi-tier business logic layer spanning yarn procurement, knitting, dyeing, cutting, sewing, finishing, accessories, sales invoicing, costing, and reporting. The system has been in production use for over a decade and accumulated several hundred SQL stored procedures, triggers, and report definitions across its modules. The application tracks every kilogram of yarn and fabric, every cutting panel, every stitched piece, every dispatch, and every rupee of value flowing through a textile manufacturer's operations from raw-material purchase to finished-goods dispatch and customer billing.

This document specifies the requirements for a modern web rebuild of the Fiberpro ERP, replacing the VB.NET desktop client with a browser-based Next.js + TypeScript + Tailwind + shadcn/ui front end, retaining the multi-tier business logic in TypeScript server code backed by a Prisma-managed SQLite database (with PostgreSQL-ready schema), and adding a built-in AI agentic harness that lets non-technical users drive the entire application through natural-language prompts. The AI agent uses the GLM-4.6 large language model with OpenAI-style function-calling, exposes typed tools across every ERP domain (orders, procurement, inventory, cutting, production, invoices, costing, HR, approvals, masters), and enforces a strict plan-then-commit safety model where every write action returns a structured plan that the human user must explicitly approve before any database mutation occurs.

The deliverable is a working, end-to-end demonstrable web application with seed data covering buyers, styles, parties, yarns, fabrics, accessories, colours, sizes, godowns, departments, employees, sales orders, purchase orders, GRNs, stock ledger, cutting orders, production entries, sales invoices, and pending approvals — coupled with an AI chat panel that can both answer read queries and orchestrate multi-step write transactions. The rebuild preserves the domain model and Indian GST / financial-year semantics of the original while modernising the user experience, accessibility, responsiveness, and audit trail.

1.1 Key Objectives

- Reproduce every functional module of the original Fiberpro ERP as web-accessible screens with equivalent data fields and workflows.
- Provide a unified AI agent layer (single chat panel) that can read from and write to every module through typed tools — turning the ERP from a form-filling application into a prompt-driven workflow.

- Enforce human-in-the-loop approval on every write action: the agent proposes a structured plan, the user reviews it on screen, and only explicit approval triggers the database commit.
- Preserve Indian textile / GST domain rules: HSN codes for fabric (5%) and garments above ₹1,000 (12%), CGST+SGST for intra-state, IGST for inter-state, financial-year 26-27, godown-based inventory, department-based production tracking.
- Ship as a self-contained Next.js project runnable in the sandbox preview, with seed data loaded through a single API call so reviewers can experiment immediately.

2. Background & Problem Statement

2.1 The Original Fiberpro ERP

The original Fiberpro is a Windows Forms .NET application distributed as a single 158 MB executable together with ~60 supporting DLLs, a 54 MB report configuration library, Stimulsoft and Crystal Reports template files for printable documents, and a SQL Server schema with hundreds of stored procedures organised into folders named SPFunction, SPQuery, and SPTriggers. The application is monolithic by design — every screen, every report, every business rule lives inside one compiled binary — and updates require shipping a new executable to every desktop. Its data model is anchored on a single StockTable master record per order × fabric × count × colour combination, and every transaction (delivery challan, GRN, multi-process GRN, return) is recorded as parent-child Trs_Del1/2/3 or Trs_Grn1/2 rows that roll up into department-wise summaries.

Reports and printed documents are first-class citizens: the report folder alone contains 80+ distinct templates for yarn DCs (delivery challans), fabric DCs, accessory GRNs, cutting-panel DCs, piece DCs, piece dispatch documents, sales invoices with GST, debit notes for yarn / fabric / accessories, transport delivery documents, packing lists, production cost reports, and one-page order status summaries. Each report has multiple variants (with-cost / without-cost, half-page / full-page, large / standard, with-image / without-image, GST / non-GST, woven / knit) reflecting years of incremental customer-driven customisation.

2.2 Why Rebuild?

Three forces motivate the rebuild. First, deployment friction: every desktop must install the executable, ODBC drivers, and Crystal Reports runtime; cross-platform and remote work are not supported. Second, integration friction: the application cannot be scripted, automated, or driven from outside; every workflow is a mouse-and-keyboard operation. Third, the rise of capable large language models with function-calling makes it possible to expose the entire ERP as a typed tool surface that an AI agent can drive — a paradigm shift that turns ERP usage from data entry into natural-language orchestration. The rebuild preserves the domain richness while moving to a web stack and adding the agent layer.

2.3 Research Basis

Requirements below are derived from three independent research streams rather than reading the original repository's documentation files (per the project brief). The first stream is inspection of the original Fiberpro artefacts at the code level — SQL stored procedure names, table names, trigger names, and report file names — to enumerate the modules, master entities, transaction types, and workflow stages the original supports. The second stream is domain knowledge of how textile and garment ERP systems work in general: yarn-to-fabric conversion, knitting-heat-setting-washing-compacting finishing pipeline, cutting-marker efficiency, bundle-based piece production tracking, godown-based inventory, and Indian GST rules for textiles. The third stream is research into AI agentic harness patterns: OpenAI function-calling, Anthropic's tool-use streaming, ReAct-style reasoning loops, plan-and-execute orchestration, and human-in-the-loop approval gates as implemented in production agent frameworks.

3. Stakeholders & Roles

The rebuild must serve the same human roles as the original desktop ERP, with role-based access control reserved as a production concern (the v1 demo runs in single-user admin mode but the schema carries a role string on every user).

Role	Primary ERP Modules	Primary Agent Use Cases
Administrator	Masters, Users, Approvals	Bulk master creation, approval workflow, audit log review
Merchandiser	Orders, Styles, Costing, Reports	Create / cancel sales orders, view order status, generate costing sheets
Storekeeper	Procurement, Inventory, GRNs	Receive GRNs, check godown stock, adjust stock, list pending POs
Production Manager	Cutting, Production, Workflow	Create cut orders, post production entries, view department-wise output
Accountant	Invoices, GST, Bills Register	Issue sales invoices with GST, view bills register, raise debit notes
HR / Payroll	HR / Employees, Departments	Maintain employee master, department master, attendance baseline
Cutting Manager	Cutting, Bundles, Barcodes	Plan cut orders, print bundle barcodes, track efficiency

Role	Primary ERP Modules	Primary Agent Use Cases
AI Agent (system)	All modules (read + write with approval)	Execute prompt-driven workflows across module boundaries

Table 1. Stakeholder roles and their primary ERP modules and AI agent use cases.

4. Functional Requirements — ERP Modules

This section enumerates the ERP functional requirements module by module. Each module is described in terms of (a) the master entities it owns, (b) the transactional operations it must support, (c) the reports or printed outputs it must produce, and (d) the cross-module interactions it has with the rest of the ERP. Requirement IDs follow the pattern FR-<MODULE>-<NNN>.

4.1 Master Data Management (FR-MASTER)

Master data is the backbone of the ERP. Every master entity has a unique human-readable code, a name, optional categorisation, audit timestamps, and is referenced by transactional tables. The rebuild must support the full master set discovered in the original schema: parties (suppliers, customers, both), buyers, buyer departments, merchandisers, seasons, exporters, styles (with HSN, SAM, category), colours, sizes, yarns, fabrics, accessories, fabric DIA, units of measure, godowns, departments, lines, employees, jobwork companies, lots, components, designs, counts, and financial-year metadata.

- FR-MASTER-001: Maintain Party master with code, name, GSTIN, PAN, address, city, state, phone, email, partyType (supplier/customer/both), opening balance. Each party is referenced from purchase orders, GRNs, sales invoices, debit notes, journals, lots, jobwork orders, and stock ledger entries.
- FR-MASTER-002: Maintain Buyer master with code, name, dept, merchandiser. Each buyer owns styles and sales orders. Optional Buyer Department sub-master for grouping.
- FR-MASTER-003: Maintain Style master with styleNo (unique), description, buyerId, category (Knit / Woven), SAM (standard allowed minutes), HSN code, seasonId, baseRate, image. Styles are referenced from sales orders, POs (where applicable), cut orders, production entries, and cost sheets.
- FR-MASTER-004: Maintain Colour and Size masters, both with sort order. Used as order line attributes and as current-stock dimension keys.
- FR-MASTER-005: Maintain Yarn, Fabric, and Accessory item masters. Yarn carries count, uom, rate, opening stock. Fabric carries dia, uom, gsm, rate, opening stock. Accessory carries uom, rate, opening stock. All three are referenced from POs, GRNs, stock ledger, and current stock.

- FR-MASTER-006: Maintain Godown master (code, name, type) — at minimum G1=Main, G2=Finished Goods, G3=Jobworker Yard. Every stock ledger entry and current stock row is scoped to a godown.
- FR-MASTER-007: Maintain Department master (code, name, orderSno, type) for production tracking — at minimum D1=Knitting, D2=Dyeing, D3=Cutting, D4=Sewing, D5=Finishing, D6=Packing. Each production entry and many stock rows carry a department foreign key.
- FR-MASTER-008: Maintain Employee master (code, name, deptId, designation, perPieceRate, monthlySalary, status). Used as operator reference on production entries and approval requester reference.
- FR-MASTER-009: Maintain Lot master (lotNo, partyId, date, type, status) for yarn and fabric lot tracking. Referenced from current stock as a dimension key.
- FR-MASTER-010: Master screens must support list view, detail view, create, edit, deactivate (soft delete), and search by code or name.

4.2 Sales Order Management (FR-ORDER)

The sales order is the central commercial artefact of the ERP — every downstream activity (PO for raw material, fabric issue, cut order, production, dispatch, invoice) is traced back to a sales order. An order carries header information (order number, buyer, style, order date, delivery date, financial year, status, totals) plus a matrix of order lines (colour × size × quantity × rate).

- FR-ORDER-001: Create a sales order with header fields orderNo (auto-assigned SO-#### if not provided or if collision detected), buyerCode, styleNo, orderDate, deliveryDate, finYear, notes; and a lines array of {colourName, sizeName, qty, rate}.
- FR-ORDER-002: On create, validate buyer exists, style exists, every colour and size referenced exists; reject the plan if any reference is missing.
- FR-ORDER-003: Compute and persist totalPcs (sum of line qty) and totalValue (sum of qty × rate) on every order header.
- FR-ORDER-004: Order status lifecycle: open → in_progress → completed → closed, plus cancelled at any point. The agent must be able to query orders by status.
- FR-ORDER-005: Cancel an order — set status to cancelled and capture a reason. Cancellation does not delete linked POs or production entries but blocks further issue against the order.
- FR-ORDER-006: List orders with include of buyer, style, and counts of lines, cut orders, and sales invoices. Detail view must eager-load lines (with colour and size), PO lines, cut orders with bundles, production entries with operator and department, sales invoices, and cost sheet.

4.3 Procurement & Inventory (FR-PROC)

Procurement covers three raw-material categories — yarn, fabric, and accessories — each with its own purchase order, GRN, stock-adjustment, opening-balance, and (where applicable) process DC and sales DC documents. The rebuild must support purchase orders against suppliers, GRN receipt against a PO (with auto stock ledger posting and current-stock upsert), stock adjustments (add / less) with reason capture, and stock queries scoped by godown.

- FR-PROC-001: Create a purchase order with poNo (auto-assigned PO-{Y|F|A|G}-### if not provided), poType (yarn|fabric|accessory|general), partyCode, orderDate, deliveryDate, finYear, notes; and a lines array of {itemType, itemCode, qty, rate}.
- FR-PROC-002: On PO create, validate party and every item exists; compute totalQty and totalValue. Auto-submit a pending Approval record so the PO follows the approval workflow before stock receipt.
- FR-PROC-003: Receive a GRN against a PO — grnNo (auto-assigned GRN-#### if not provided), poNo, godownCode, receivedQty, optional partyDcRef and deptCode. The receive action atomically: creates the GRN with one line, posts a stock ledger entry (txnType=purchase_grn) with inKgs or inPcs depending on itemType, upserts current stock for that godown, updates PO status to received or partial, and increments the PO line receivedQty.
- FR-PROC-004: Stock adjustment (add / less) — adjust the current stock of an item at a godown with a reason; posts a stock ledger entry with txnType=stock_adjustment_add or stock_adjustment_less.
- FR-PROC-005: Query current stock with optional godown filter. Each current stock row carries itemType, itemId, godownId, lotId, colourId, sizeId, deptId, orderId, kgs, mtrs, pcs, rate.
- FR-PROC-006: List the most recent 30 stock ledger entries (the transaction-level audit trail) with godown and party include, for the inventory view.
- FR-PROC-007: List all GRNs against a PO with line detail.

4.4 Cutting & Bundle Management (FR-CUT)

Cutting converts issued fabric into cutting panels grouped into bundles with barcodes. The cut order records the fabric issued (in kgs), total pieces cut, marker length, number of plies, and computed efficiency. Bundle barcodes enable downstream sewing-line tracking.

- FR-CUT-001: Create a cut order — cutNo (auto-assigned CUT-#### if not provided), orderNo, fabricIssued (kgs), totalPcs, markerLength, noOfPlies, efficiency, cutDate.
- FR-CUT-002: On cut order create, auto-generate bundles of up to 100 pieces each, with bundleNo = {cutNo}/B{n} and a Code-39-style barcode string suitable for printing.
- FR-CUT-003: List cut orders with order, buyer, style, and bundles include. Each cut order status: planned → in_progress → completed.

- FR-CUT-004: Track bundle status through in_cutting → issued_to_sewing → in_sewing → completed → rejected.

4.5 Production Tracking (FR-PROD)

Production entries record operator output by department, by date, by bundle, and by order. Each entry carries a quantity, a per-piece rate, the operator (employee), the department, the production line, and an amount. Production entries roll up into department-wise summaries (today's output, week-to-date, order-to-date) and feed the costing module.

- FR-PROD-001: Post a production entry — orderNo, deptCode, prodDate, bundleNo, operatorCode, qty, rate, optional styleNo, colourName, sizeName, lineId. Compute amount = qty × rate.
- FR-PROD-002: List the 100 most recent production entries with operator, department, order, buyer, style include.
- FR-PROD-003: List production lines with their owning department.
- FR-PROD-004: Department-wise summary with count of production entries for each department, ordered by department orderSno.

4.6 Sales Invoices & GST (FR-INV)

Sales invoices are tax-compliant commercial documents. Each invoice carries the issuing party, the linked order, bill type (sales, jobwork, yarn_sales, fab_sales), total quantity, taxable value, GST rate and type (CGST+SGST for intra-state, IGST for inter-state), the computed GST amount, the bill total, and an issued / cancelled status. The rebuild supports the same GST computation rules as the original — split CGST and SGST equally when the type is cgst_sgst, full IGST when the type is igst.

- FR-INV-001: Create a sales invoice — invoiceNo (auto-assigned INV-#### if not provided), orderNo, partyCode, billType, totalQty, taxableValue, gstRate, gstType (cgst_sgst | igst), invoiceDate, notes. Compute cgstRate, sgstRate, igstRate, cgstAmt, sgstAmt, igstAmt, billAmount.
- FR-INV-002: Indian GST defaults — fabric HSN 5%, garments > ₹1,000 HSN 12%, accessories HSN 18%. The agent's system prompt carries these defaults so it can suggest them when the user does not specify a rate.
- FR-INV-003: List sales invoices with party and order include, ordered by invoice date descending.
- FR-INV-004: Invoice lifecycle: issued → paid → cancelled. The schema supports status transitions; the agent can issue but cannot directly mark paid (that requires an accounting-side journal entry, reserved for v2).

4.7 Costing & Budget (FR-COST)

Costing compares budgeted per-piece cost (yarn, fabric, accessories, processing, labour, overheads) against actuals accumulated from procurement and production. The original ERP runs an SP_BudAndActual stored procedure family producing style-wise and order-wise actual-vs-budget reports; the rebuild provides a CostSheet table per order and a dashboard-style costing view.

- FR-COST-001: Maintain a CostSheet per order with budgeted yarn, fabric, accessories, processing, labour, overhead amounts and an actual-totals column populated by aggregation.
- FR-COST-002: List cost sheets with order, buyer, style include.
- FR-COST-003: Compute the variance = (actual − budget) per cost head and per piece.

4.8 HR & Payroll (FR-HR)

HR maintains the employee master and department master, and provides the basis for production-entry operator reference and approval requester reference. Full payroll calculation (per-piece earnings, monthly salary, overtime, deductions) is reserved for v2; v1 ships the master maintenance and the read-only summary view.

- FR-HR-001: Maintain Employee master (code, name, deptId, designation, perPieceRate, monthlySalary, status, joinDate).
- FR-HR-002: List employees with department include, ordered by code.
- FR-HR-003: List departments ordered by orderSno for the department master view.

4.9 Approvals (FR-APPR)

The approval workflow tracks pending requests for write-sensitive entities. The original ERP's approval model is implicit (POs require sign-off before receipt); the rebuild makes it explicit by creating an Approval record whenever a write-sensitive document is created via the agent.

- FR-APPR-001: Create an Approval record on PO creation — entity=po, entityId=poId, step=1, requestedBy=agent, status=pending.
- FR-APPR-002: Approve a pending request — set status=approved, approvedBy, approvedAt. The referenced entity (PO, etc.) is then unlocked for downstream operations.
- FR-APPR-003: List pending approvals with enriched entity detail (e.g., the full PO with party and lines).

4.10 Dashboard (FR-DASH)

The dashboard is the landing screen. It aggregates six KPIs (open orders, pending POs, stock value, today's production pieces, pending approvals, open invoices) and lists the five most recent orders, POs, cut orders, and invoices for quick navigation.

- FR-DASH-001: Compute KPIs in a single round-trip: count orders with status in (open, in_progress), count POs with status in (open, partial), sum (kgs + mtrs + pcs) × rate across all current stock, sum qty across production entries whose prodDate is today, count pending approvals, count sales invoices with status=issued.
- FR-DASH-002: Each KPI is clickable and navigates the user to the corresponding module view.

5. Functional Requirements — AI Agent Harness

The AI agentic harness is the differentiating feature of the rebuild. It is not a chatbot bolted on top of the ERP; it is a typed tool surface over the entire ERP that an LLM drives via function-calling, with a human-in-the-loop approval gate on every write. This section specifies the harness's architecture, tool catalogue, prompt protocol, safety rules, and audit trail.

5.1 Architecture

The harness is a thin server-side loop that bridges the user's chat panel and the LLM provider. The front end posts the conversation history to /api/agent as a Server-Sent-Events stream; the server appends a system prompt, instantiates an OpenAI-compatible client pointed at the GLM-4.6 endpoint, and runs a multi-step agent loop. Each step sends the accumulated message history with the typed tool catalogue; if the model returns text, it is streamed back as text-delta events; if the model returns tool_calls, each call is executed against the typed tool registry and the result is fed back into the next step. The loop terminates when the model stops calling tools or after a hard cap of six steps.

5.2 Tool Catalogue

Every tool is implemented as a TypeScript object with a name, a domain tag, an isWrite boolean, a Zod-validated argument schema, and an async execute function. Read tools return immediately with structured JSON. Write tools return a plan object containing a human-readable summary, a list of creates / updates / side-effects, and a closure-captured commit function that performs the actual database mutation when called by the approval endpoint.

5.2.1 Read Tools

Tool Name	Domain	Purpose
list_orders	orders	List sales orders, optional status filter
get_order	orders	Single order detail with lines, cuts, production, invoices, cost
list_purchase_orders	procurement	List POs with party and line/grn counts
get_purchase_order	procurement	Single PO detail with lines and GRNs
list_buyers	masters	List buyer master
list_styles	masters	List style master
list_parties	masters	List party master (suppliers / customers)
list_yarns / list_fabrics / list_accessories	masters	List item masters
list_colours / list_sizes	masters	List colour and size masters
list_godowns / list_departments	masters	List godown and department masters
list_employees	hr	List employee master
get_stock	inventory	Current stock by godown, optional itemType filter
get_stock_ledger	inventory	Recent stock ledger entries
list_cut_orders	cutting	List cut orders with bundles
get_production_status	production	Production entries roll-up by order, by department, by date
list_invoices	accounting	List sales invoices with party and order
list_cost_sheets	costing	List cost sheets with variance
list_pending_approvals	approvals	List pending approval records
get_dashboard	dashboard	KPIs + recent activity
list_agent_turns	audit	Recent agent turn audit log

Table 2. Read-only tools exposed to the AI agent. Read tools execute immediately and auto-mark their audit record as approved.

5.2.2 Write Tools

Tool Name	Domain	Plan Summary
create_order	orders	Create sales order + lines (header + matrix)
cancel_order	orders	Set order status=cancelled with reason
create_purchase_order	procurement	Create PO + lines; auto-create pending Approval
receive_grn	procurement	Atomic: GRN + stock ledger + current stock upsert + PO status update + PO line receivedQty increment
adjust_stock	inventory	Add or reduce current stock with reason
create_cut_order	cutting	Create cut order + auto-generate bundles with barcodes
post_production_entry	production	Post production entry (operator output)
create_sales_invoice	accounting	Create sales invoice with GST computation
approve_pending	approvals	Approve a pending approval record (unlocks entity)

Table 3. Write tools. Every write tool returns a plan and a closure-captured commit function; only explicit user approval triggers commit.

5.3 Prompt Protocol & Safety Rules

The system prompt instructs the LLM to behave as the Fiberpro Agent, an AI assistant embedded in the ERP. The prompt encodes six non-negotiable safety rules that govern every interaction: (1) read prompts call read tools immediately and synthesise a bullet-point answer; (2) write prompts first call any required read tools to validate references (e.g., list_buyers, list_styles), then call the write tool, then explicitly tell the user the action is awaiting approval in the chat panel — and never claim the action is done until the commit result is observed; (3) if a referenced entity does not exist, list the relevant masters first; (4) Indian GST rules apply (CGST+SGST for intra-state, IGST for inter-state, with the standard textile HSN rates); (5) Indian number formatting (₹, lakhs, crores) is used; (6) financial year 26-27 and the standard godown (G1/G2/G3) and department (D1-D6) codes are baked in.

An additional rule tells the model that for every create tool the document number (orderNo, poNo, grnNo, invoiceNo, cutNo) is optional and auto-assigned server-side — the model should not

pass these fields unless the user explicitly demands a specific number. This eliminates the most common agent failure (creating SO-1001 when SO-1001 already exists) by making the server the source of truth for sequential numbering.

5.4 Plan-Then-Commit Safety Model

The plan-then-commit model is the harness's central safety guarantee. Every write tool's execute function returns an object with: text (human-readable one-liner for the chat), plan (structured summary, creates array, updates array, sideEffects array), and a closure-captured commit function that performs the actual database mutation. The agent route never calls commit; it streams the plan back to the chat panel. The chat panel renders the plan in an amber-highlighted card with two buttons: Approve & Commit and Reject. Only when the user clicks Approve & Commit does the front end call /api/agent/approve with the toolName and args, which re-executes the tool to regenerate the plan and then calls commit().

This design has three important properties. First, the LLM never has direct database access — it only ever returns a plan describing what would happen. Second, the user sees a structured, machine-readable summary of the proposed mutation before any database write occurs. Third, the commit function is regenerated at approval time from the same args, which means any drift between plan-time and commit-time state (e.g., a concurrent edit) is detected because execute will throw if a referenced entity no longer exists.

5.5 Audit Trail

Every tool call — read or write — is persisted in the AgentTurn table with the user's prompt, the JSON-stringified plan (if any), the JSON array of tool calls with their names and args, a result snippet (capped at 2,000 characters), an approved flag (false for writes until the user approves), and a userId. The agent turns are exposed via the erp?resource=agent_turns API and visible in the audit view, giving administrators a complete record of every AI-driven action.

6. Non-Functional Requirements

6.1 Performance

Dashboard KPI round-trip must complete in under 200 ms against a seeded database; the inventory, orders, and procurement list endpoints must return in under 100 ms. The agent loop must reach a first text-delta within 2 seconds and complete a typical 2-step tool call within 8 seconds. Production postings and GRN receipts must execute atomically (db.$transaction) so partial failures cannot leave stock and PO state out of sync.

6.2 Security

V1 ships in single-user admin mode for sandbox demonstration. The schema carries a User table with role string (admin | merchandiser | storekeeper | accountant | production_mgr | hr | cutting_mgr) and an AgentTurn.userId field ready for production authentication. The agent approve endpoint must accept only the toolName and args (never arbitrary SQL or code); the tool registry is the only entry point to database mutations.

6.3 Reliability & Data Integrity

All multi-table writes (GRN receipt, cut-order + bundle generation, order + lines) must run inside a Prisma transaction so that a failure at any step rolls back the entire operation. Unique constraints on orderNo, poNo, grnNo, invoiceNo, cutNo enforce business-key uniqueness. The current-stock compound unique key (itemType + itemId + godownId + lotId + colourId + sizeId + deptId + orderId) prevents duplicate stock rows.

6.4 Usability & Accessibility

The UI must be fully responsive (mobile-first with desktop enhancement), keyboard-navigable, and screen-reader friendly using semantic HTML and ARIA roles. The agent panel must open via ⌘+K / Ctrl+K shortcut. Toast notifications confirm every user-initiated action. Long lists use max-height with custom scrollbar styling. The sidebar collapses to a Sheet on mobile widths.

6.5 Maintainability

All code is TypeScript with strict typing. The agent tools file is the single source of truth for the tool catalogue; adding a tool requires only appending an object to the readTools or writeTools array. The ERP API follows a single resource router pattern (GET /api/erp?resource=X). Database schema changes flow through Prisma migrate; the seed script is re-runnable and idempotent.

6.6 Localisation

All currency values use Indian number formatting (₹, lakhs, crores). All dates default to ISO format but the UI presents them in dd MMM yy style. The agent's system prompt bakes in Indian GST rates, financial-year 26-27, and the standard godown and department codes. The ERP ships English-only in v1; the schema and prompts are structured to permit a Hindi or Tamil localisation layer in v2.

7. System Architecture

7.1 Layered Architecture

The rebuild follows a four-layer architecture. The presentation layer is a Next.js 16 App Router single-page application using React 19, Tailwind CSS 4, and the shadcn/ui component library (New York style). The API layer exposes two endpoints: /api/erp (REST-style GET with a resource query parameter for all ERP reads) and /api/agent (SSE-streaming POST for the AI agent loop), plus /api/agent/approve (POST for commit) and /api/seed (POST for re-seeding the demo database). The domain layer lives in src/lib/agent/tools.ts (the agent tool catalogue, which contains the bulk of business validation and mutation logic) and the route handlers themselves. The persistence layer is Prisma ORM over SQLite for v1, schema-compatible with PostgreSQL for production.

7.2 Component Map

Layer	Component	Responsibility
Presentation	src/app/page.tsx	Single-page shell with sidebar + 11 view slots + agent sheet
Presentation	src/components/erp/*.tsx	Per-module views (dashboard, orders, procurement, inventory, cutting, production, invoices, costing, hr, workflow, masters)
Presentation	src/components/agent/agent-panel.tsx	AI chat panel with SSE consumer, tool-call rendering, pending-approval UI
API	src/app/api/erp/route.ts	Single GET router for all ERP reads (dashboard, orders, POs, inventory, cutting, production, invoices, costing, hr, approvals, masters, agent_turns)
API	src/app/api/agent/route.ts	SSE-streaming agent loop (max 6 steps, OpenAI-compatible function calling against GLM-4.6)
API	src/app/api/agent/approve/route.ts	Approval endpoint — re-executes tool, calls commit()
API	src/app/api/seed/route.ts	Re-seed endpoint — shells out to scripts/seed.ts
Domain	src/lib/agent/tools.ts	Typed tool catalogue (read + write) with Zod schemas and plan-then-commit pattern

Layer	Component	Responsibility
Domain	src/lib/db.ts	Prisma client singleton
Persistence	prisma/schema.prisma	Schema for 30+ tables covering masters, transactions, stock, agent audit
Persistence	scripts/seed.ts	Idempotent demo data seeder

Table 4. Component map across the four architecture layers.

7.3 Data Model Highlights

The schema covers the same domain as the original Fiberpro schema, normalised for a relational store. Highlights: the StockTable master of the original becomes the CurrentStock table keyed on (itemType, itemId, godownId, lotId, colourId, sizeId, deptId, orderId). The Trs_Del1/2/3 and Trs_Grn1/2 parent-child transaction tables of the original become flattened GRN, GRNLine, CutOrder, CutBundle, ProductionEntry tables — preserving the parent-child semantics but simplifying queries. The master tables Mas_Buyer, Mas_Color, Mas_Component, Mas_Count, Mas_Dept, Mas_Design, Mas_Dia, Mas_Emp, Mas_Fabric, Mas_Fcy, Mas_JobWrkComp become Buyer, Colour, Component, Count, Department, Design, Dia, Employee, Fabric, Party, JobworkCompany. The full schema is in prisma/schema.prisma.

7.4 Technology Stack

Concern	Technology	Rationale
Framework	Next.js 16 App Router	Single-page shell + API routes + Turbopack HMR
Language	TypeScript 5	Strict typing across the entire stack
UI components	shadcn/ui (New York) + Radix primitives + Lucide	Accessible, headless, themeable, no lock-in
Styling	Tailwind CSS 4 + tw-animate-css	Utility-first, responsive, dark-mode ready
State (client)	Zustand + TanStack Query	Lightweight global state + server cache
Database	Prisma ORM over SQLite (schema PG-ready)	Type-safe queries, migrations, single-file dev DB
LLM provider	GLM-4.6 via OpenAI-compatible SDK	Function calling, 200K context, fast streaming
Schema validation	Zod 4 + zod-to-json-schema	Single source of truth for tool argument schemas

Concern	Technology	Rationale
Notifications	Sonner + Radix Toast	Stacked toast feedback for every action
Charts	Recharts	Dashboard mini-charts (KPI trends, dept output)

Table 5. Technology stack with rationale for each choice.

8. Key Workflows

8.1 Order-to-Invoice (Happy Path)

A merchandiser opens the agent panel and types 'Create a sales order for buyer B001, style S-1001, 5000 pcs at ₹350/pc (Red/M=1000, Red/L=1000, Blue/M=1500, Blue/L=1500), delivery 2026-10-15'. The agent calls list_buyers, list_styles, then create_order. The tool validates the buyer and style exist, resolves the colours and sizes, computes totalPcs=5000 and totalValue=₹17,50,000, picks the next free orderNo (SO-####), and returns a plan describing the order header + 5 line rows. The chat panel renders the plan in an amber card with Approve & Commit / Reject buttons. The merchandiser clicks Approve; the server re-executes the tool, calls commit(), inserts the order and lines, and refreshes the dashboard. The new SO-#### appears in the open-orders KPI count.

8.2 Yarn Procurement to Stock

A storekeeper types 'Create a yarn PO for SUP001, 500 kg of YRN-30s cotton at ₹180/kg, delivery 2026-09-05'. The agent calls create_purchase_order with poType=yarn, partyCode=SUP001, lines=[{itemType:yarn, itemCode:YRN-30s, qty:500, rate:180}]. The tool validates the party and yarn, resolves the uom, computes totalQty=500 and totalValue=₹90,000, picks the next free PO-Y-###, and returns a plan. Approval creates the PO and a pending Approval record. Once the approval is unlocked, the storekeeper can type 'Receive 500 kg of PO-Y-### into godown G1' — the agent calls receive_grn, which atomically creates the GRN, posts a stock ledger entry with inKgs=500, upserts current stock for that yarn at G1, and updates the PO status to received.

8.3 Cutting to Production

A cutting manager types 'Create a cut order for SO-1001, 200 kg fabric, 4000 pcs, marker 4.5m, 800 plies, 88% efficiency'. The agent calls create_cut_order. The tool resolves the order, picks the next free CUT-####, and returns a plan that creates the cut order plus 40 auto-generated bundles (each up to 100 pcs) with barcodes. After approval, the production manager can post production entries bundle-by-bundle — 'Post 95 pcs for SO-1001, bundle CUT-0001/B3, operator E001, sewing line D4, rate ₹18/pc'. Each production entry rolls into the department-wise summary on the production dashboard.

8.4 Sales Invoice with GST

An accountant types 'Create a sales invoice for SO-1005, customer CUST-DEL, 1500 pcs, taxable ₹5,25,000, GST 12% cgst_sgst' (intra-state). The agent calls create_sales_invoice with gstType=cgst_sgst and gstRate=12. The tool computes cgstRate=6, sgstRate=6, cgstAmt=₹31,500, sgstAmt=₹31,500, billAmount=₹5,88,000. After approval, the invoice is issued and shows up in the open-invoices KPI. If the customer were inter-state, the user would specify gstType=igst and the tool would compute the full 12% as IGST.

9. Constraints, Assumptions, Dependencies

9.1 Constraints

- The web rebuild cannot directly execute the original VB.NET desktop binary; it reimplements the modules in TypeScript against the same domain model.
- The original's Crystal Reports and Stimulsoft report templates cannot be opened in the browser; v1 ships an on-screen tabular report view rather than pixel-perfect printable documents. v2 will add a printable report layer via Puppeteer or React-pdf.
- v1 ships single-user admin mode (no authentication) to keep the sandbox demo frictionless; production deployment requires NextAuth.js or equivalent session middleware, plus role-based tool gating in the agent.
- The agent runs against the GLM-4.6 model via an internal Z.AI endpoint; production deployment must provide its own OpenAI-compatible endpoint and API key.

9.2 Assumptions

- Financial year is 26-27 (1 April 2026 to 31 March 2027). All orders, POs, GRNs, and invoices default to this finYear.
- Three godowns (G1, G2, G3) and six departments (D1-D6) suffice for the demo. Production deployments can extend these.
- Indian GST rules apply: 5% on fabric, 12% on garments above ₹1,000, 18% on accessories. The system prompt carries these as defaults.
- Sequential document numbers are sufficient (SO-####, PO-{Y|F|A|G}-###, GRN-####, INV-####, CUT-####). Production deployments may switch to per-financial-year sequences.

9.3 Dependencies

- Node.js ≥ 20 and Bun ≥ 1.3 for the development runtime.
- Next.js 16, React 19, Prisma 6, Tailwind 4, shadcn/ui (New York variant), Zod 4, OpenAI SDK 7.

- GLM-4.6 model access via an OpenAI-compatible chat-completions endpoint with function-calling support.

10. Acceptance Criteria

The deliverable is accepted when all the following are demonstrably true in the sandbox preview:

- AC-01: The / route renders a dashboard with six populated KPIs and four recent-activity lists without any console or runtime errors.
- AC-02: All 11 sidebar views (dashboard, orders, procurement, inventory, cutting, production, invoices, costing, hr, workflow, masters) render real seeded data with no empty tables or loading spinners.
- AC-03: The Re-seed demo data button rebuilds the database and the dashboard reflects fresh counts within 5 seconds.
- AC-04: The Agent panel opens via the Sparkles button and via ⌘+K, and displays six suggested prompts on first open.
- AC-05: A read prompt ('List all open purchase orders') triggers a tool call, returns structured JSON, and synthesises a bullet-point answer in under 8 seconds.
- AC-06: A write prompt ('Create a sales order for buyer B001, style S-1001, 5000 pcs at ₹350/pc …') triggers list_buyers + list_styles + create_order and renders a structured plan card with Approve & Commit and Reject buttons.
- AC-07: Clicking Approve & Commit persists the order to the database; the dashboard open-orders KPI increments within 1 second; the agent turns audit log records the approved turn.
- AC-08: Document numbers are auto-assigned server-side when the LLM does not pass them; collisions trigger the next free sequential number rather than a unique-constraint error.
- AC-09: The agent route persists every tool call (read or write) as an AgentTurn row with prompt, plan, toolCalls, result, approved flag, and userId.
- AC-10: The layout is responsive — sidebar collapses to a Sheet on mobile widths; the agent panel occupies the full screen on mobile and a 2xl-width sheet on desktop.
- AC-11: `bun run lint` passes with zero errors (warnings for unused eslint-disable directives are acceptable).
- AC-12: All write tools run multi-table mutations inside a Prisma transaction (db.$transaction) so partial failures roll back cleanly.

11. Future Scope (Out of v1)

- Authentication and role-based access control via NextAuth.js, with role-gated tool surfaces (e.g., only accountants can call create_sales_invoice).

- Printable document generation — yarn DC, fabric DC, accessory GRN, piece DC, sales invoice, debit notes, packing list — via Puppeteer + React-pdf, matching the original's Stimulsoft / Crystal Reports layout family.
- Production migration to PostgreSQL with proper indexing on StockTable compound key and partitioning of StockLedger by finYear.
- Multi-step agent orchestration with persistent state (e.g., 'Generate a costing report for all open orders and email it to finance@...') — v1 caps the agent loop at 6 steps with no background-task state.
- WebSocket-based real-time notifications so multiple users see stock changes and approval state changes without manual refresh.
- Hindi / Tamil localisation layer over the UI and the agent system prompt.
- Production audit log enrichment — diff of every committed record (before/after JSON) stored alongside AgentTurn, with replay capability.